

BASE DE DATOS: SISTEMA ESCOLAR

-- Crear base de datos del colegio

```
CREATE DATABASE IF NOT EXISTS colegio_db;
```

```
USE colegio_db;
```

-- 1. Tabla estudiantes

```
CREATE TABLE estudiantes (  
    id_estudiante INT PRIMARY KEY AUTO_INCREMENT,  
    matricula VARCHAR(20) UNIQUE NOT NULL,  
    nombre VARCHAR(100) NOT NULL,  
    apellido VARCHAR(100) NOT NULL,  
    fecha_nacimiento DATE,  
    email VARCHAR(150),  
    telefono VARCHAR(20),  
    fecha_inscripcion DATE DEFAULT (CURRENT_DATE),  
    activo BOOLEAN DEFAULT TRUE  
);
```

-- 2. Tabla profesores

```
CREATE TABLE profesores (  
    id_profesor INT PRIMARY KEY AUTO_INCREMENT,  
    codigo_profesor VARCHAR(20) UNIQUE NOT NULL,  
    nombre VARCHAR(100) NOT NULL,  
    apellido VARCHAR(100) NOT NULL,  
    especialidad VARCHAR(100),  
    email VARCHAR(150),  
    telefono VARCHAR(20),  
    fecha_contratacion DATE DEFAULT (CURRENT_DATE)  
);
```

-- 3. Tabla cursos

```
CREATE TABLE cursos (  
    id_curso INT PRIMARY KEY AUTO_INCREMENT,  
    codigo_curso VARCHAR(20) UNIQUE NOT NULL,  
    nombre VARCHAR(100) NOT NULL,  
    descripcion TEXT,  
    creditos INT DEFAULT 1,  
    id_profesor INT,  
    cupo_maximo INT DEFAULT 30,  
    activo BOOLEAN DEFAULT TRUE,  
    FOREIGN KEY (id_profesor) REFERENCES profesores(id_profesor)  
);
```

-- 4. Tabla inscripciones

```
CREATE TABLE inscripciones (  
    id_inscripcion INT PRIMARY KEY AUTO_INCREMENT,  
    id_estudiante INT NOT NULL,  
    id_curso INT NOT NULL,  
    fecha_inscripcion DATE DEFAULT (CURRENT_DATE),  
    calificacion DECIMAL(4,2) CHECK (calificacion BETWEEN 0 AND 10),  
    estado ENUM('Inscrito', 'Aprobado', 'Reprobado', 'Retirado') DEFAULT 'Inscrito',
```

```
FOREIGN KEY (id_estudiante) REFERENCES estudiantes(id_estudiante),  
FOREIGN KEY (id_curso) REFERENCES cursos(id_curso),  
UNIQUE KEY unique_inscripcion (id_estudiante, id_curso)  
);
```

-- 5. *Tabla auditoria_calificaciones*

```
CREATE TABLE auditoria_calificaciones (  
  id_auditoria INT PRIMARY KEY AUTO_INCREMENT,  
  id_inscripcion INT,  
  calificacion_anterior DECIMAL(4,2),  
  calificacion_nueva DECIMAL(4,2),  
  diferencia DECIMAL(4,2),  
  usuario VARCHAR(100),  
  fecha_cambio DATETIME DEFAULT CURRENT_TIMESTAMP,  
  motivo VARCHAR(200)  
);
```

-- *Insertar datos en estudiantes (más de 10)*

```
INSERT INTO estudiantes (matricula, nombre, apellido, fecha_nacimiento, email,  
telefono) VALUES  
( 'EST2024001', 'Ana', 'García', '2008-05-15', 'ana@colegio.edu', '600111222'),  
( 'EST2024002', 'Carlos', 'Martínez', '2007-08-22', 'carlos@colegio.edu', '600222333'),  
( 'EST2024003', 'María', 'López', '2008-03-10', 'maria@colegio.edu', '600333444'),  
( 'EST2024004', 'Pedro', 'Sánchez', '2007-11-30', 'pedro@colegio.edu', '600444555'),  
( 'EST2024005', 'Laura', 'Fernández', '2008-01-25', 'laura@colegio.edu', '600555666'),  
( 'EST2024006', 'Javier', 'Gómez', '2007-07-18', 'javier@colegio.edu', '600666777'),  
( 'EST2024007', 'Elena', 'Díaz', '2008-09-05', 'elena@colegio.edu', '600777888'),  
( 'EST2024008', 'David', 'Ruiz', '2007-12-12', 'david@colegio.edu', '600888999'),  
( 'EST2024009', 'Sofía', 'Hernández', '2008-04-20', 'sofia@colegio.edu', '600999000'),  
( 'EST2024010', 'Miguel', 'Torres', '2007-10-08', 'miguel@colegio.edu', '600000111'),  
( 'EST2024011', 'Carmen', 'Vázquez', '2008-06-14', 'carmen@colegio.edu', '611111222'),  
( 'EST2024012', 'Alejandro', 'Ramírez', '2007-02-28', 'alejandro@colegio.edu',  
'622222333');
```

-- *Insertar datos en profesores (más de 10)*

```
INSERT INTO profesores (codigo_profesor, nombre, apellido, especialidad, email)  
VALUES  
( 'PROF001', 'Juan', 'Rodríguez', 'Matemáticas', 'juan@colegio.edu'),  
( 'PROF002', 'Marta', 'Gil', 'Lengua', 'marta@colegio.edu'),  
( 'PROF003', 'Roberto', 'Castro', 'Ciencias', 'roberto@colegio.edu'),  
( 'PROF004', 'Isabel', 'Navarro', 'Historia', 'isabel@colegio.edu'),  
( 'PROF005', 'Francisco', 'Molina', 'Educación Física', 'francisco@colegio.edu'),  
( 'PROF006', 'Teresa', 'Ortega', 'Inglés', 'teresa@colegio.edu'),  
( 'PROF007', 'Alberto', 'Serrano', 'Informática', 'alberto@colegio.edu'),  
( 'PROF008', 'Patricia', 'Reyes', 'Arte', 'patricia@colegio.edu'),  
( 'PROF009', 'Ricardo', 'Méndez', 'Música', 'ricardo@colegio.edu'),  
( 'PROF010', 'Silvia', 'Cortés', 'Filosofía', 'silvia@colegio.edu'),  
( 'PROF011', 'Fernando', 'Santos', 'Geografía', 'fernando@colegio.edu'),  
( 'PROF012', 'Beatriz', 'Pérez', 'Biología', 'beatriz@colegio.edu');
```

-- *Insertar datos en cursos (más de 10)*

INSERT INTO cursos (codigo_curso, nombre, descripcion, creditos, id_profesor, cupo_maximo) VALUES

('MAT101', 'Matemáticas Básicas', 'Álgebra y aritmética fundamental', 4, 1, 25),
('LEN201', 'Lengua Española', 'Gramática y literatura', 3, 2, 30),
('CIE301', 'Ciencias Naturales', 'Biología y física básica', 3, 3, 28),
('HIS401', 'Historia Universal', 'Historia mundial', 2, 4, 35),
('EDF501', 'Educación Física', 'Deportes y salud', 2, 5, 40),
('ING601', 'Inglés Básico', 'Inglés nivel A1-A2', 3, 6, 25),
('INF701', 'Informática', 'Ofimática y programación básica', 4, 7, 20),
('ART801', 'Arte y Dibujo', 'Técnicas artísticas', 2, 8, 15),
('MUS901', 'Música', 'Teoría musical y práctica', 2, 9, 20),
('FIL1001', 'Filosofía', 'Pensamiento crítico', 2, 10, 30),
('GEO1101', 'Geografía', 'Geografía mundial', 2, 11, 25),
('BIO1201', 'Biología Avanzada', 'Biología celular', 4, 12, 20);

-- Insertar datos en inscripciones (más de 10)

INSERT INTO inscripciones (id_estudiante, id_curso, calificacion, estado) VALUES

(1, 1, 8.5, 'Aprobado'),
(1, 2, 7.0, 'Aprobado'),
(1, 3, 9.0, 'Aprobado'),
(2, 1, 6.5, 'Aprobado'),
(2, 4, 8.0, 'Aprobado'),
(2, 5, 9.5, 'Aprobado'),
(3, 2, 5.5, 'Reprobado'),
(3, 6, 7.5, 'Aprobado'),
(3, 7, 8.0, 'Aprobado'),
(4, 1, 9.0, 'Aprobado'),
(4, 3, 8.5, 'Aprobado'),
(4, 8, 6.0, 'Aprobado'),
(5, 4, 7.5, 'Aprobado'),
(5, 9, 8.0, 'Aprobado'),
(5, 10, 9.0, 'Aprobado'),
(6, 5, 6.5, 'Aprobado'),
(6, 11, 7.0, 'Aprobado'),
(6, 12, 8.5, 'Aprobado'),
(7, 6, 9.0, 'Aprobado'),
(7, 7, 5.0, 'Reprobado'),
(8, 8, 7.5, 'Aprobado'),
(8, 9, 8.0, 'Aprobado'),
(9, 10, 6.0, 'Aprobado'),
(9, 11, 9.5, 'Aprobado'),
(10, 12, 8.0, 'Aprobado'),
(10, 1, 7.5, 'Aprobado'),
(11, 2, 8.0, 'Aprobado'),
(12, 3, 9.0, 'Aprobado');

-- Insertar algunos registros en auditoria

INSERT INTO auditoria_calificaciones (id_inscripcion, calificacion_anterior, calificacion_nueva, usuario, motivo)
VALUES (1, 7.5, 8.5, 'PROF001', 'Revisión de examen');

 ENUNCIADOS - SISTEMA ESCOLAR**Pregunta 1: Procedimiento Almacenado**

La dirección del colegio necesita un reporte detallado del rendimiento académico por profesor y especialidad para evaluar la efectividad docente. El procedimiento debe generar un informe que muestre métricas como promedio de calificaciones, tasa de aprobación, número de estudiantes por curso y carga de créditos. Esta información es crucial para la asignación de recursos, reconocimientos docentes y mejora continua del programa educativo. Se requieren al menos 2 JOINS para relacionar profesores, cursos e inscripciones.

Crea un procedimiento llamado `sp_reporte_rendimiento_profesores` que:

1. Reciba como parámetro el año actual (asumir inscripciones del año actual)
2. Genere un reporte que incluya:
 - Nombre completo del profesor
 - Especialidad
 - Número de cursos impartidos
 - Total de estudiantes inscritos
 - Promedio general de calificaciones
 - Porcentaje de aprobación (estudiantes aprobados / total estudiantes)
 - Total de créditos impartidos

CALL `sp_reporte_rendimiento_profesores(2024);`

Pregunta 2: Función

Para el sistema de becas, necesitamos una función que calcule el índice académico de cada estudiante basándose en sus calificaciones y los créditos de los cursos. El índice se calcula como la suma de (calificación × créditos) dividida entre la suma de créditos. Este índice determina la elegibilidad para becas, reconocimientos académicos y participación en programas especiales. La función debe manejar estudiantes sin calificaciones y considerar solo cursos aprobados.

Crea una función llamada `fn_calcular_indice_academico` que:

1. Reciba el ID de un estudiante (INT)
2. Calcule el índice académico usando la fórmula:
$$\text{SUM}(\text{calificacion} * \text{creditos}) / \text{SUM}(\text{creditos})$$
3. Considerar solo cursos con estado 'Aprobado'
4. Si el estudiante no tiene cursos aprobados, retornar 0.00
5. Si el estudiante no existe, retornar NULL

Ejemplo de llamada:

```
SELECT nombre, fn_calcular_indice_academico(id_estudiante) AS indice
FROM estudiantes WHERE activo = TRUE ORDER BY indice DESC;
```

Pregunta 3: Trigger

Para garantizar que no se exceda el cupo máximo de estudiantes en un curso, necesitamos un trigger que verifique la disponibilidad antes de permitir una nueva inscripción. Este trigger previene sobrepoblación en las aulas y asegura una experiencia educativa de calidad. Debe contar los estudiantes ya inscritos (excluyendo retirados) y comparar con el cupo máximo del curso. Si no hay cupo disponible, debe rechazar la inscripción automáticamente.

Crea un trigger llamado `tg_verificar_cupo_curso` que:

1. Se active BEFORE INSERT en la tabla inscripciones
2. Cuente el número actual de estudiantes inscritos en el curso (excluyendo estado 'Retirado')
3. Compare con el cupo máximo del curso
4. Si el cupo está lleno:
 - Establecer el estado de la inscripción a 'Retirado' automáticamente
 - NO permitir la inserción normal
5. Si hay cupo disponible, permitir la inserción con estado 'Inscrito'
6. Asegurar que un estudiante no se pueda inscribir dos veces al mismo curso

Ejemplo de prueba:

```
SELECT COUNT(*) FROM inscripciones WHERE id_curso = 1 AND estado != 'Retirado';
```

-- Intentar inscribir estudiante nuevo cuando el cupo esté lleno

```
INSERT INTO inscripciones (id_estudiante, id_curso)
VALUES (12, 1);
```

Pregunta 4: Trigger

Para mantener la integridad académica, necesitamos monitorear cambios significativos en las calificaciones. Este trigger detectará modificaciones sospechosas (aumentos o disminuciones drásticas) y registrará una alerta en auditoría. Consideraremos sospechoso cualquier cambio mayor a 2 puntos o cambios que conviertan un estado de 'Suspendido' a 'Aprobado' o viceversa. Esto ayuda a prevenir errores administrativos y posibles irregularidades en la evaluación.

Crea un trigger llamado `tg_auditar_cambios_calificacion` que:

1. Se active AFTER UPDATE en la tabla inscripciones
2. Detecte cambios en la calificación (cuando `NEW.calificacion != OLD.calificacion`)
3. Verifique si el cambio es sospechoso según:
 - Diferencia absoluta > 2 puntos
 - Cambio que altere el estado académico (de 'Suspendido' a 'Aprobado' o viceversa)
4. Si es sospechoso:
 - Registrar en `auditoria_calificaciones` con motivo descriptivo
 - Incluir diferencia calculada
5. Usar 'SISTEMA' como usuario
6. Actualizar automáticamente el estado según la nueva calificación:
 - `calificacion >= 5.0` → 'Aprobado'
 - `calificacion < 5.0` → 'Suspendido'

Ejemplo de prueba:

-- *Cambiar calificación drásticamente (de 5.5 a 8.5)*

UPDATE inscripciones SET calificacion = 8.5

WHERE id_inscripcion = 3;

-- *Verificar alerta en auditoría*

SELECT * FROM auditoria_calificaciones ORDER BY fecha_cambio DESC LIMIT 1;